

DP & State Transitions Notes

Authored by: Arpit Raj, LNMIIT Jaipur

Date: July 24, 2026

© 2026 Arpit Raj. All rights reserved.

Concept & Reasoning

Since each row has only 3 cells, we can precompute all valid row patterns. only 12 rows satisfy

cell11 != cell12

cell12 != cell13

Then,

dp[i][j]

means

Number of ways to paint the first $i+1$ rows such that the i -th row uses the j -th valid pattern.

To transition,

the current row itself is already valid,

we only need to check whether each column differs from the previous row.

So,

previousPattern



currentPattern

Column1 : different

Column2 : different

Column3 : different

If all three columns differ, we can transition.

Solution Implementation (C++)

```
class Solution {
public:
    int numOfWays(int n) {
        const int MOD=1e9+7;

        vector<vector<int>> pattern;

        //we need to store valid patterns that a cell can take such
that no 2 adjacent subcell have same colour
        for(int i=0;i<3;i++){
            for(int j=0;j<3;j++){
                if(i==j) continue;

                for(int k=0;k<3;k++){
                    if(j==k) continue;

                    pattern.push_back({i,j,k});
                }
            }
        }

        int k=pattern.size();
        vector<vector<long long>> dp(n,vector<long long>(k,0));

        //dp[i][j] is basically storing ways to paint ith cell with
jth pattern
        //base case , first cell can take any pattern

        for(int j=0;j<k;j++){
            dp[0][j]=1;
        }

        for(int i=1;i<n;i++){ //painting remaining cells

            for(int prev=0;prev<k;prev++){
                for(int curr=0;curr<k;curr++){

                    if(pattern[prev][0]==pattern[curr][0] ||
                       pattern[prev][1]==pattern[curr][1] ||
                       pattern[prev][2]==pattern[curr][2]){
                        continue;
                    } //maan lo column wise same nikal gaya
                     //RGB AND RBG , R AND R matching so invalid

                    dp[i][curr]+=dp[i-1][prev];
                }
            }
        }
    }
};
```

```
        dp[i][curr]=dp[i][curr]%MOD;

    }

}

int ans=0;
for(int j=0;j<k;j++){
    ans+=dp[n-1][j];
    ans=ans%MOD;
}

return ans;

}

};
```